A2 · It Is Always DNS (Except When It Is Not)

CS425 Computer Networks · in class activity · graded pass/fail

TURN THIS IN ON PAPER
before you leave the room
A3 is built on this

Names _____ Date _____

_____ _____

TARGET CARD · copy these from the board before you touch anything

ALPHA = _____ BRAVO = _____

YOUR FOUR PROBES

Set up your terminal, once

All three are **typed into your terminal**, not saved into a file, and last only as long as that window is open. New tab means all three again.

1. Store the two addresses. Use the numbers from your target card, not the ones printed here. No spaces around the **=**.

```
ALPHA=203.0.113.10
BRAVO=203.0.113.42
PowerShell instead:  $ALPHA = "203.0.113.10"
```

2. Make a shortcut for the long curl command. Type this as **one line** and press Enter. Nothing gets printed, which is correct: you have just taught the shell a new word.

```
probe() { curl -sS -o /dev/null -m 8 -w 'connect=%{time_connect} total=%{time_total} code=%
{http_code}\n' "http://$1:$2/"; }
```

If your prompt turns into a bare > you missed a quote or a brace and the shell is waiting for the rest. Press **Ctrl-C** and type the line again. This is the single most common way this step goes wrong.

3. Test it before you trust it.

```
probe $ALPHA 8080
connect=0.022526 total=0.065459 code=200
```

`command not found` means step 2 did not take; do it again. `probe` takes the address then the port, so `probe $ALPHA 8081` is station 2 and `probe $BRAVO 8080` asks the other machine.

One piece of syntax worth knowing. `$ALPHA` means "the address I stored in step 1". In `dig`, the `@` is **not part of the address**: it means "ask *this* server", so `@$ALPHA` reads as "put this question to the ALPHA machine". You have to say that here because `cs425.lab` is not a real internet name, so ALPHA is the only machine that knows the answer.

QUESTION	TYPE THIS, EXACTLY	WINDOWS
Does the name resolve? two forms, see below	`dig +short alpha.cs425.lab @$ALPHA -p 5353` `dig alpha.cs425.lab @$ALPHA -p 5353`	`nslookup -port=5353 alpha.cs425.lab 203.0.113.10`
Does the host answer ICMP?	`ping -c 3 $ALPHA`	`ping -n 3 $ALPHA`
Is the port open?	Linux **`nc -vz -w 5`** `$ALPHA 8080` macOS **`nc -vz -G 5`** `$ALPHA 8080`	`Test-NetConnection $ALPHA -Port 8080`
Does the service work?	`curl -sS -o /dev/null -m 8 -w 'connect=%{time_connect} total=%{time_total} code=% {http_code}\n' http://$ALPHA:8080/`	

Shown against station 1 and against `alpha.cs425.lab`, a name that does resolve. For other stations change the **port** (or the **name**), and use `$BRAVO` where the station says so.

Both forms of dig matter. `+short` prints *only the answer records*, so a name with no answer prints **nothing at all** and still exits successfully. That is not a broken command; the empty line *is* the answer. Whenever you get one, re-run without `+short` and read the `status:` field. Station 4 is exactly this case.

Read what curl prints, and time it. Failing in milliseconds and hanging for seconds are different failures. curl words the fast one differently per platform (`Connection refused` on Linux, `Couldn't connect to server` on macOS); `nc -vz` names it outright on both. Use the right

nc timeout flag above, or it sits for 75 seconds printing nothing. **Keep `-p 5353` on every dig.** Terms: glossary on the back.

ROUND 1 · THE HEALTHY SIGNATURE

You cannot recognize broken until you have looked at working. Probe ALPHA port 8080 with all four commands.

#	TARGET	DIG SAID	PING SAID	CURL PRINTED, WORD FOR WORD	SECONDS TO ANSWER
1	`ALPHA 8080` `baseline`				

Answer before you move on

What did ping report as the minimum round trip time, what did curl report for `connect=`, and why are those two numbers in the same neighborhood?

ROUND 2 · FOUR WAYS TO FAIL

Do them in order, the order is the argument. Stations 2 and 3 are the same host, the same laptop and the same network, and they behave in opposite ways.

#	TARGET	DIG SAID	PING SAID	CURL PRINTED, WORD FOR WORD	SEC TO FAIL	THE CAUSE, IN YOUR WORDS	LAYER
2	`ALPHA 8081`						
3	`ALPHA 8082`						
4	`ghost` `.cs425.lab`						
5	`mirage` `.cs425.lab`						

The one distinction that matters

Stations 2 and 3 are the two failures you will meet for the rest of your career, and they are opposites. One means a machine received your packet and answered “no”. The other means nobody answered anything at all.

Which is which, and **what did the host send back** in the case that failed fast? It is one TCP flag.

ROUND 2, CONTINUED · THE TWO DNS STATIONS

Station 4 · `ghost.cs425.lab`

```
dig ghost.cs425.lab @$ALPHA -p 5353     # no +short, you need the status line
```

With `+short` this prints nothing, which is a real answer badly displayed. What was in the `status:` field, and is it even *possible* for this to be a connectivity problem? How many packets did you send to the target?

Station 5 · `mirage.cs425.lab`

```
dig +short mirage.cs425.lab @$ALPHA -p 5353    # this one does answer
```

This looks identical to station 2 from where you are sitting, and it is not the same failure at all. Name the **one probe** that separates them, and say what is special about the address dig handed you.

ROUND 3 · THE PROBE THAT LIES

Ping BRAVO. Then curl BRAVO port 8080. One fails, one succeeds, and the one that fails is the one most people would have used to decide the host was dead.

#	TARGET	DIG SAID	PING SAID	CURL PRINTED, WORD FOR WORD	SEC TO FAIL	THE CAUSE, IN YOUR WORDS	LAYER
6	`BRAVO 8080`						

1. Which probe reported the machine unreachable, and which one proved it was not?
2. What protocol does ping use, and at which layer does that protocol live?
3. ALPHA answers ping and BRAVO does not, yet both serve the same page on 8080. What is different between the two hosts, and does that difference live *on* the host or *in front of* it?
4. Write one sentence you could say to a coworker who has just told you “the server is down, it does not ping”.

IF YOU FINISH EARLY

Drop the `-p 5353` from one of your dig commands and run it again. If the answer changes, your network is intercepting DNS and answering in place of the server you asked for. Write down both answers: that is a middlebox lying to you, the same class of bug as station 5.

Graded pass/fail on a serious attempt at every round. Hand this in before you leave, with every name at the top. You get it back for A3, which is built on your station table.

NAMES

DNS
The application layer protocol that turns a name into an IP address. A real query to a real server, so it can fail like anything else.

Resolver
The server your machine asks. Handed to you by the network you joined, which means that network chooses who answers your questions.

Zone
A slice of the name space one set of servers is responsible for, like `cs425.lab`.

Authoritative
Holding the real records rather than a cached copy. An authoritative "no" is a fact.

A record
Maps a name to an IPv4 address.

NXDOMAIN
That name does not exist. An answer, not an error: the server was reachable and told you there is nothing there. You have sent **zero** packets to your target, so this is not a connectivity problem. `dig +short` hides this by printing an empty line; drop `+short` to see it.

ADDRESSES

IP address
Network layer identifier for an *interface*, not a machine and not a user.

Private address, RFC 1918
`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. **Not routable on the public internet.** Packets sent to one from outside its network go to your own gateway and die there.

NAT
Rewrites addresses and ports so many private machines share one public address. Why nobody can open a connection *to* your laptop.

CIDR, `/16`
An address plus how many leading bits are fixed. `132.178.0.0/16` covers `132.178.0.0` through `132.178.255.255`.

GETTING THERE

RTT
Round trip time. Nothing completes faster than one RTT, and most things take several.

ICMP
The network layer protocol for control and error messages, including ping's echo request and reply. Rides directly in IP **alongside** TCP and UDP, so it has no port numbers.

ping
Sends an ICMP echo request. Success proves reachability. **Failure proves almost nothing**, because filtering ICMP while allowing TCP is extremely common.

CONNECTIONS

TCP
Reliable ordered byte stream. Sets up a connection before any data moves.

Port
A 16 bit number saying *which program* gets the data. The address finds the machine, the port finds the process.

Listening socket
A socket a server is waiting on. No listener means the host has nobody to hand an arriving connection to.

Three way handshake
SYN, SYN-ACK, ACK. `connect()` returns when the SYN-ACK arrives, so connecting costs **one round trip** before any of your data moves.

RST
A TCP flag meaning "nothing here, stop", sent when a connection arrives for a port with no listener. Getting one is **informative**: it proves the address is routable, the host is up, and its stack is running.

Connection refused
What a client reports after a RST. About one round trip, so effectively instant.

Connection timeout
What a client reports after **nothing at all**. It retransmits with exponential backoff and gives up on its own clock, which is why a drop takes seconds and a refusal takes milliseconds.

FILTERING

Firewall, packet filter
Anything that inspects packets and decides whether they pass, usually on addresses and ports.

DROP versus REJECT
DROP discards silently; the sender waits out its own timeout. **REJECT** sends back an error, usually a RST. A dropped packet and a host that does not exist look identical from the client, which is exactly the point.

Security group
AWS's packet filter, attached to the virtual interface rather than the host. Defaults to DROP, and sits **in front of** the machine, so a blocked packet never reaches the OS at all.

Middlebox
Anything on the path doing more than forwarding: firewalls, NAT, proxies, DNS interceptors. Why one protocol behaves differently on two networks.

CURL EXIT CODES

`0 / 6 / 7 / 28`
Worked, could not resolve, failed to connect, timed out. **7 versus 28 is the pair that matters:** 7 means something answered and said no, 28 means nothing answered.

`connect=`
`0.000000` means the handshake never completed. A real value followed by a timeout means TCP succeeded and the *application* never answered.

THE FOUR LAYERS

Application
Do the useful thing. HTTP, SMTP, DNS.

Transport
Bytes to the right *program*. TCP, UDP.

Network
A packet to the right *host*. IP, ICMP.

Link
A frame to the next hop on this wire. Ethernet, Wi-Fi.